

**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана**
**(национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

# ЛАБОРАТОРНАЯ РАБОТА №3-4

«Функциональные возможности языка Python.»

**ДИСЦИПЛИНА:**
**«Парадигмы и конструкции языков программирования»**

Выполнил: студент гр. ИУ5-32Б _____ (Краснова А. А.)
(Подпись) (Ф.И.О.)

Проверил: _____ ( Гапанюк Ю. Е. )
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

**Цель лабораторной работы:** изучение возможностей функционального программирования в языке Python.

**Задача 1 (файл field.py)**

Необходимо реализовать генератор field. Генератор field последовательно выдает значения ключей словаря.

В качестве первого аргумента генератор принимает список словарей, дальше через *args генератор принимает неограниченное количествово аргументов.

- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно None, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно None, то оно пропускается. Если все поля содержат значения None, то пропускается элемент целиком.

Код программы:

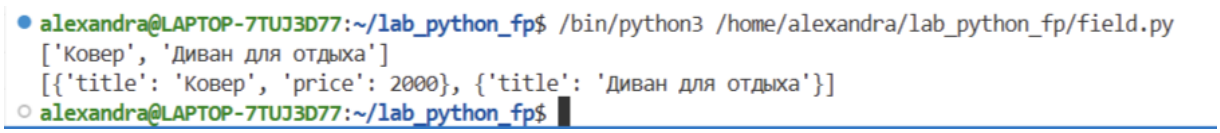
```
# Пример:
# goods = [
#    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#    {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price':
2000}, {'title': 'Диван для отдыха', 'price': 5300}

def field(items, *args):
    assert len(args) > 0
    for item in items:
        if len(args) == 1:
            # Передано одно поле - выдаём только его значение
            value = item.get(args[0])
            if value is not None:
                yield value
        else:
            # Передано несколько полей - выдаём словарь
            result = {}
            for key in args:
                value = item.get(key)
                if value is not None:
                    result[key] = value
            if len(result) > 0:
                yield result


if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'}
    ]
```

```
    print(list(field(goods, 'title')))
    print(list(field(goods, 'title', 'price')))
```

Вывод программы:




**Задача 2 (файл gen_random.py)**
Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:
gen_random(5, 1, 3) должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Код программы:

```
import random


# Пример:
# gen_random(5, 1, 3) должен выдать выдать 5 случайных чисел
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1
def gen_random(num_count, begin, end):
    for i in range(num_count):
        yield random.randint(begin, end)

if __name__ == '__main__':
    print(list(gen_random(5, 1, 3)))
```

Вывод программы:




**Задача 3 (файл unique.py)**

- Необходимо реализовать итератор Unique(данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный bool-параметр ignore_case, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен False.

- При реализации необходимо использовать конструкцию **kwargs.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Код программы:

```
from gen_random import gen_random
# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.ignore_case = kwargs.get('ignore_case', False)
        self.seen = set()  # уже встречавшиеся значения

    def __next__(self):
        while True:
            # Когда элементы закончатся, next() сам выбросит StopIteration
            item = next(self.items)
            key = item
            if self.ignore_case and isinstance(item, str):
                key = item.lower()
            if key not in self.seen:
                self.seen.add(key)
                return item

    def __iter__(self):
        return self


if __name__ == '__main__':
    data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
    print(list(Unique(data)))

    data = gen_random(10, 1, 3)
    print(list(Unique(data)))

    data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
    print(list(Unique(data)))
    print(list(Unique(data, ignore_case=True)))
```

Вывод программы:

```
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$ /bin/python3 /home/alexandra/lab_python_fp/unique.py
[1, 2]
[3, 2, 1]
['a', 'A', 'b', 'B']
['a', 'b']
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$
```

**Задача 4 (файл sort.py)**

Дан массив 1, содержащий положительные и отрицательные числа. Необходимо **одной строкой кода** вывести на экран массив 2, которые

содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции sorted.
Пример:
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
Необходимо решить задачу двумя способами:

1. С использованием lambda-функции.
2. Без использования lambda-функции.

Код программы:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

Вывод программы:

```
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$ /bin/python3 /home/alexandra/lab_python_fp/sort.py
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$
```

**Задача 5 (файл print_result.py)**
Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводить в столбик через знак равенства.

Код программы:

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)
        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f'{key} = {value}')
```

```
        else:
            print(result)
        return result
    return wrapper


@print_result
def test_1():
    return 1


@print_result
def test_2():
    return 'iu5'


@print_result
def test_3():
    return {'a': 1, 'b': 2}


@print_result
def test_4():
    return [1, 2]


if __name__ == '__main__':
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Вывод программы:

```
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$ /bin/python3 /home/alexandra/lab_python_fp/print_result.py
!!!!!!!!
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
```

**Задача 6 (файл cm_timer.py)**

Необходимо написать контекстные менеджеры cm_timer_1 и cm_timer_2, которые считают время работы блока кода и выводят его на экран. cm_timer_1 и cm_timer_2 реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки contextlib).

Код программы:

```
import time
from contextlib import contextmanager


# Способ 1: на основе класса
class cm_timer_1:
    def __enter__(self):
        self.start = time.time()

    def __exit__(self, exc_type, exc_value, traceback):
        print('time:', time.time() - self.start)


# Способ 2: с использованием библиотеки contextlib
@contextmanager
def cm_timer_2():
    start = time.time()
    yield
    print('time:', time.time() - start)


if __name__ == '__main__':
    with cm_timer_1():
        time.sleep(5.5)

    with cm_timer_2():
        time.sleep(5.5)
```

Вывод программы:

```
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$ /bin/python3 /home/alexandra/lab_python_fp/cm_timer.py
time: 5.500175952911377
time: 5.500243186950684
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$
```

**Задача 7 (файл process_data.py)**

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле data_light.json содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - f1, f2, f3, f4. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора @print_result печатается результат, а контекстный менеджер cm_timer_1 выводит время работы цепочки функций.
- Предполагается, что функции f1, f2, f3 будут реализованы в одну строку. В реализации функции f4 может быть до 3 строк.
- Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка

должна игнорировать регистр. Используйте наработки из предыдущих задач.

- Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию filter.
- Функция f3 должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию map.
- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист C# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Код программы:

```
import json
import sys
from field import field
from gen_random import gen_random
from unique import Unique
from print_result import print_result
from cm_timer import cm_timer_1

# Путь к файлу передаётся при запуске: python process_data.py data_light.json
path = sys.argv[1]

with open(path, encoding='utf-8') as f:
    data = json.load(f)


@print_result
def f1(arg):
    return sorted(Unique(field(arg, 'job-name'), ignore_case=True), key=lambda x:
x.lower())


@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))


@print_result
def f3(arg):
    return list(map(lambda x: x + ' с опытом Python', arg))


@print_result
```

```
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f'{job}, зарплата {salary} руб.' for job, salary in zip(arg,
salaries)]


if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Запуск:
cd ~/lab_python_fp
python3 process_data.py data_light.json

Вывод программы:

```
Юрисконсульт. Контрактный управляющий
Юрист
Юрист (специалист по сопровождению международных договоров, английский - разговорный)
Юрист волонтер
Юристконсульт
f2
Программист
Программист / Senior Developer
Программист 1С
Программист C#
Программист C++
Программист C++/C#/Java
Программист/ Junior Developer
Программист/ технический специалист
Программистр-разработчик  информационных систем
f3
Программист с опытом Python
Программист / Senior Developer с опытом Python
Программист 1С с опытом Python
Программист C# с опытом Python
Программист C++ с опытом Python
Программист C++/C#/Java с опытом Python
Программист/ Junior Developer с опытом Python
Программист/ технический специалист с опытом Python
Программистр-разработчик  информационных систем с опытом Python
f4
Программист с опытом Python, зарплата 178047 руб.
Программист / Senior Developer с опытом Python, зарплата 102139 руб.
Программист 1С с опытом Python, зарплата 128377 руб.
Программист C# с опытом Python, зарплата 196140 руб.
Программист C++ с опытом Python, зарплата 122550 руб.
Программист C++/C#/Java с опытом Python, зарплата 125299 руб.
Программист/ Junior Developer с опытом Python, зарплата 152583 руб.
Программист/ технический специалист с опытом Python, зарплата 191808 руб.
Программистр-разработчик  информационных систем с опытом Python, зарплата 198059 руб.
time: 0.08652353286743164
alexandra@LAPTOP-7TUJ3D77:~/lab_python_fp$
```